

Full Stack Development with MERN

Project Documentation format

1. Introduction

- **Project Title:** Implement Client Script and UI Policy (Incident)
- **Team Members:** List team members and their roles.

Name	Role	Work done
Mahalakshmi R	Team Leader	Requirements and final testing
Jebeena R	ServiceNow Developer	UI Policy creation
Gayathri K	ServiceNow Developer	Client Script creation
Kumari K	Tester / Documentation	Test cases and documentation

2. Project Overview

- **Purpose:** The purpose of this project is to make the Incident form in ServiceNow smarter and easier to use. UI Policies and Client Scripts control how the form behaves in the browser, so agents enter correct data and make fewer mistakes.
- **Features:** Key features and functionalities.
 - o Mandatory fields based on a condition (e.g. Configuration item is mandatory when Category is Hardware)
 - o Show or hide fields based on Category or State
 - o Read-only fields after the Incident is saved
 - o onChange Client Script that shows a message when Priority is 1 - Critical
 - o onSubmit Client Script that stops saving when the Short description is too short
 - o Update Set to move the changes from Development to Production

3. Architecture

- **Frontend:** The frontend is the Incident form in the ServiceNow Web UI. UI Policies and Client Scripts run in the user's browser using JavaScript and the g_form API. UI Policy is used for simple rules (visible, mandatory, read-only), and Client Script is used for custom logic.

Example onChange Client Script (Priority):

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {  
    if (isLoading || newValue === '') { return; }  
    if (newValue == '1') {  
        g_form.showFieldMsg('priority', 'Critical incident: please inform the Incident Manager.', 'info');  
    } else {  
        g_form.hideFieldMsg('priority');  
    }  
}
```

Example onSubmit Client Script (Short description):

```
function onSubmit() {
    var desc = g_form.getValue('short_description');
    if (desc.length < 10) {
        g_form.addErrorMessage('Short description must have at least 10 characters.');
```

- **Backend:** The backend is the ServiceNow application server (Now Platform). It uses server-side JavaScript. Important rules are repeated in a Business Rule (using GlideRecord), because client-side checks can be skipped when records are created by import or API.
- **Database:** Incident records are stored in the incident table of the ServiceNow database. When the agent saves the form, the platform inserts or updates the record. Main fields used in this project:

Field (column name)	Type	Use in this project
Caller (caller_id)	Reference	Shows who reported the issue
Category (category)	Choice	Condition for UI Policy
Configuration item (cmdb_ci)	Reference	Mandatory when Category is Hardware
Priority (priority)	Choice	Used in the onChange Client Script
Short description (short_description)	String	Checked in the onSubmit Client Script
State (state)	Choice	Shows Close code and Close notes when Resolved

4. Setup Instructions

- **Prerequisites:** List software dependencies.
 - o ServiceNow Personal Developer Instance (free from developer.servicenow.com)
 - o User with the admin role
 - o Web browser (Google Chrome or Microsoft Edge)
 - o Basic knowledge of JavaScript
- **Installation:** Step-by-step guide to set up the instance and the environment.
 1. Sign up at developer.servicenow.com and request a Personal Developer Instance.
 2. Log in to the instance with the admin user.
 3. Create a new Update Set (System Update Sets > Local Update Sets) and make it current, so all changes are saved together.
 4. Open the Incident form (Incident > Create New) to check the default behavior.
 5. Create the UI Policy and Client Scripts as given in the next sections. No environment variables or extra installation are needed.

5. Folder Structure

- **Client:** In ServiceNow there are no folders for files. The client-side records are stored in tables and opened from the left navigation menu.

```

System UI
└─ UI Policies          -> Incident - Configuration item mandatory
                           (with UI Policy Actions)
System Definition
└─ Client Scripts       -> Incident - Priority onChange
                           -> Incident - Short description onSubmit

```

- **Server:** The server-side records and the packaging of changes.

```

System Definition
└─ Business Rules       -> Incident - Validate short description
System Security
└─ Access Control (ACL) -> Field access by role
System Update Sets
└─ Local Update Sets    -> Client Script and UI Policy (Incident)

```

6. Running the Application

- Provide commands to start the application locally.
 - o **Frontend:** Log in to the instance and open Incident > Create New. The UI Policy and Client Scripts load with the form. (No npm command is needed, as ServiceNow is already running on the cloud.)
 - o **Backend:** Click Submit on the form. The Business Rule and database update run on the server automatically.
 - o To test, change Category to Hardware, change Priority to 1 - Critical, and try to save with a short description.

7. API Documentation

- Document all endpoints exposed by the backend. Client Scripts and UI Policies run inside the form, so they have no endpoints. The Incident data can be used by other tools through the ServiceNow Table API.

Method	Endpoint	Parameters	Description
GET	/api/now/table/incident	sysparm_limit, sysparm_query	Get a list of incidents
GET	/api/now/table/incident/{sys_id}	sys_id	Get one incident
POST	/api/now/table/incident	JSON body with fields	Create an incident
PATCH	/api/now/table/incident/{sys_id}	sys_id, JSON body	Update an incident

Example request (POST) body:

```

{
  "caller_id": "Abel Tuter",
  "category": "hardware",
  "short_description": "Laptop screen not working"
}

```

Example response:

```

{
  "result": {
    "number": "INC0010023",

```

```
"category": "hardware",
"short_description": "Laptop screen not working",
"state": "1"
}
```

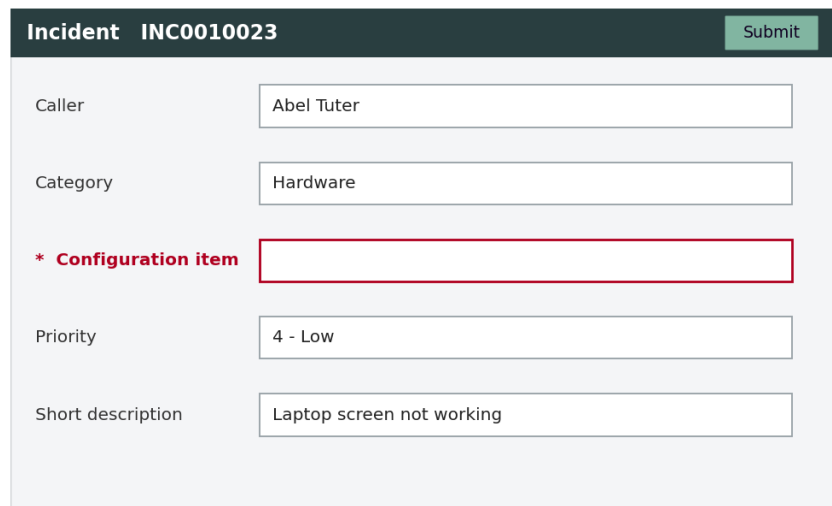
8. Authentication

- Explain how authentication and authorization are handled in the project.
- **Authentication:** The user logs in to ServiceNow with a user ID and password (Single Sign-On can also be used). After login, the browser keeps a session cookie, and the platform checks it for each request.
- **Authorization:** Roles (like itil and admin) and Access Control Lists (ACLs) decide who can see or change a field. Client Scripts and UI Policies only change the form view, so ACLs are used for real security.
- **API access:** The Table API uses Basic Authentication or OAuth tokens. Inside the form, the session token is used by the platform.

9. User Interface

- Screenshots showing the different UI features.

UI Policy: Configuration item becomes mandatory when Category is Hardware.



The screenshot shows a ServiceNow incident form titled "Incident INC0010023" with a "Submit" button. The form contains several fields: "Caller" (Abel Tuter), "Category" (Hardware), "Priority" (4 - Low), and "Short description" (Laptop screen not working). The "Configuration item" field is highlighted with a red border and a red asterisk, indicating it is mandatory due to a UI Policy triggered by the "Category" field being set to "Hardware".

Figure 1: UI Policy makes the Configuration item field mandatory

Client Script: Message appears when Priority is set to 1 - Critical.

Incident INC0010023
Submit

Caller
Abel Tuter

Category
Hardware

* Configuration item
Dell Latitude 7420

Priority
1 - Critical

Critical incident: please inform the Incident Manager.

Short description
Email server is down for all users

Figure 2: onChange Client Script shows a field message

10. Testing

- Testing strategy and tools used.
- **Strategy:** Manual testing on a new Incident and an existing Incident, with different values and different users. Each rule is tested for both the true and false condition.
- **Tools:** Browser Developer Tools (Console), Script Debugger, Impersonate User, and the Automated Test Framework (optional).

No.	Test case	Expected result	Status
1	Select Category = Hardware and leave Configuration item empty	Configuration item is mandatory; form cannot be saved	Pass
2	Select Category = Software	Configuration item is not mandatory	Pass
3	Change Priority to 1 - Critical	Message to inform the Incident Manager is shown	Pass
4	Enter Short description with less than 10 characters and save	Error message shown; form is not saved	Pass
5	Set State to Resolved	Close code and Close notes become mandatory	Pass

11. Screenshots or Demo

- Screenshot of the application working:

The screenshot shows a web form titled "Incident INC0010023" with a "Submit" button. Below the header, a red-bordered box contains the error message: "Error Message: Short description must have at least 10 characters." The form fields are as follows:

Field	Value
Caller	Abel Tuter
Category	Software
Priority	3 - Moderate
Short description	Help

Figure 3: onSubmit Client Script stops saving and shows an error message

- Demo link: <https://drive.google.com/drive/folders/1XgWoaBVbjw-6UnU6BiVTFIxsy6aQ7IIO>

12. Known Issues

- Client-side rules do not work when records are created by import, email, or API. A Business Rule is needed for these cases.
- UI Policy and Client Script may conflict if both change the same field, so the order must be checked.
- Some Client Script features may not work the same way on the mobile app or in List view.
- Too many mandatory fields can slow down agents during urgent incidents.

13. Future Enhancements

- Use Data Policies to apply the same rules for imports and APIs.
- Use Catalog Client Scripts and UI Policies for the Service Catalog forms.
- Add more dynamic rules, such as auto-filling the Assignment group based on Category.
- Use Predictive Intelligence to suggest the Category and Priority.
- Add Automated Test Framework (ATF) tests to check the rules after every upgrade.